

Лабораторные работы 1 и 2

Теория для изучения и повторения

ЛР 1 · ИПС «Арахна»

Информационно-поисковая система по документам локальной сети. Вариант 3: элементы ИИ в интерфейсе, векторная стратегия, русский язык.

TF-IDF · косинусная мера · морфология · Рокчио · метрики РОМИП

ЛР 2 · «Толмач»

Автоматическое распознавание языка HTML-документов. Вариант 4: русский и немецкий.

N-граммы Кавнара — Тренкла · алфавитный метод · персептрон на nupru · голосование

Составлено по методическим указаниям, исходному коду, отчётам и результатам обеих лабораторных работ.
Все численные примеры пересчитаны функциями из кода систем.

Содержание

Часть 0. Общая картина: что объединяет две работы.....	4
0.1. Две системы бок о бок.....	4
0.2. Общий конвейер обработки текста.....	4
0.3. Общая архитектура обеих систем.....	5
Часть 1. ЛР 1 — информационно-поисковая система «Арахна».....	6
1.1. Постановка задачи.....	6
1.2. Базовые понятия информационного поиска.....	6
1.3. Из чего состоит поисковая система.....	7
1.4. Весовые коэффициенты: TF-IDF.....	8
1.5. Модели и стратегии поиска.....	9
1.6. Векторная модель: TF-IDF и косинусная мера.....	10
1.7. Элементы ИИ в интерфейсе (суть варианта 3).....	11
1.8. Оценка качества: метрики РОМИП.....	12
1.9. Структура приложения «Арахна».....	15
1.10. Где это применяется в жизни.....	17
Часть 2. ЛР 2 — распознавание языка текста, система «Толмач».....	18
2.1. Постановка задачи.....	18
2.2. Основные понятия.....	18
2.3. Три шага распознавания.....	19
2.4. Обзор методов распознавания.....	20
2.5. Метод N-грамм (Кавнар — Тренкл, 1994).....	21
2.6. Алфавитный метод.....	22
2.7. Нейросетевой метод.....	23
2.8. Голосование методов и уверенность.....	24
2.9. Оценка и эксперименты.....	24
2.10. Структура приложения «Толмач».....	26
2.11. Где это применяется в жизни.....	28
Часть 3. Технологии и компоненты.....	29
3.1. Сквозные архитектурные принципы.....	29
Часть 4. Шпаргалка формул.....	31
Часть 5. Вопросы для самопроверки.....	32
5.1. По ЛР 1.....	32
5.2. По ЛР 2.....	33
5.3. Сквозные вопросы.....	34
Часть 6. Глоссарий.....	36
Источники.....	37

Как пользоваться конспектом

Конспект построен от общего к частному. **Часть 0** показывает, что объединяет обе работы, — прочитайте её первой, она задаёт каркас. **Части 1 и 2** разбирают каждую работу: понятия, формулы с разобранными примерами, устройство приложения, результаты и применение в жизни. **Часть 3** — технологии и архитектурные принципы. **Части 4–6** — для повторения: шпаргалка формул, вопросы для самопроверки с ответами и русско-английский глоссарий.

Материал размечен цветными выносками — по ним удобно пробегать глазами при повторении:

ОПРЕДЕЛЕНИЕ

строгое определение — то, что стоит уметь произнести близко к тексту методички.

НА ПАЛЬЦАХ

объяснение простыми словами или аналогией — чтобы понять, а не зазубрить.

РАЗОБРАННЫЙ ПРИМЕР

численный пример, который стоит пересчитать руками хотя бы раз.

В НАШЕЙ СИСТЕМЕ

как это устроено и какие цифры получились в наших системах.

ПОДВОХ

частая ошибка или тонкое место, о котором любят спрашивать.

В ЖИЗНИ

где та же идея работает в реальных продуктах.

План повторения за вечер

1. Часть 0 — 10 минут: общая схема «образ → сравнение → решение».
2. ЛР 1: формулы (1.5), (1.6), косинус; пересчитать пример из раздела 1.6 и метрики из раздела 1.8.
3. ЛР 2: три шага, три метода; пересчитать out-of-place и алфавитное расстояние; понять, зачем нейросети фрагменты разной длины.
4. Шпаргалка формул — прочитать и закрыть; воспроизвести по памяти.
5. Вопросы для самопроверки — отвечать вслух, не подглядывая.

Часть 0. Общая картина: что объединяет две работы

Обе лабораторные работы решают одну и ту же задачу, только с разных сторон: **текст превращается в поисковый образ** — набор признаков с весами, образы сравниваются мерой близости или расстояния, и по результату сравнения принимается решение. В ЛР 1 образ документа сравнивается с образом запроса, чтобы упорядочить документы. В ЛР 2 образ документа сравнивается с образом языка, чтобы выбрать язык. Если держать в голове эту общую схему, все частные понятия ложатся на свои места.

0.1. Две системы бок о бок

	ЛР 1 · ИПС «Арахна»	ЛР 2 · «Толмач»
Задача	найти и упорядочить документы по запросу	определить язык документа
Тип задачи	ранжирование (информационный поиск)	классификация на 2 класса
Что сравнивается	ПОД ↔ ПОЗ (документ и запрос)	ПОД ↔ ПОЯ (документ и язык)
Признаки	леммы — слова в нормальной форме	символьные N-граммы, буквы
Вес признака	$A_{ij} = Q_{ij} \cdot B_i$ (TF-IDF)	ранг по частоте или относительная частота
Мера	косинус угла: больше — ближе	расстояние: меньше — ближе
Результат	упорядоченный список документов	один язык + голосование методов
Оценка качества	метрики РОМИП по эталонной разметке: P, R, MAP, nDCG, bpref	доля верных ответов, матрица ошибок, время
Вариант	3: интерфейс с пользователем, ЛВС, векторная стратегия, русский	4: русский и немецкий, HTML, методы N-грамм, алфавитный, нейросетевой
Стек	Python, FastAPI + Jinja2, SQLite, pymorphy3	Python, FastAPI + Jinja2, numpy, профили в JSON

0.2. Общий конвейер обработки текста

Почти любая система обработки естественного языка проходит одни и те же этапы. Удобно учить обе работы, раскладывая их по этой лестнице:

- 1. Получение документа** — паук обходит сеть (ЛР 1) или читается файл коллекции (ЛР 2).
- 2. Извлечение текста из формата** — HTML, DOCX, PDF превращаются в чистый текст: теги, скрипты и стили отбрасываются.
- 3. Нормализация** — регистр, небуквенные символы, приведение Unicode к одной форме.
- 4. Разбиение на единицы** — токены (слова) или символьные N-граммы.
- 5. Отбор и приведение** — отбрасываются стоп-слова, слова приводятся к лемме (ЛР 1).
- 6. Построение образа** — вектор весов (ЛР 1) или профиль с рангами и частотами (ЛР 2).
- 7. Сравнение с эталоном** — с образом запроса или с образом языка.
- 8. Решение** — упорядочить выдачу или выбрать язык.
- 9. Оценка качества** — сравнение ответов системы с разметкой, сделанной человеком.

НА ПАЛЬЦАХ · как объяснить за 30 секунд

ЛР 1 — библиотекарь: вы задаёте вопрос своими словами, он приносит стопку книг, самые подходящие сверху, и объясняет, почему взял каждую.

ЛР 2 — человек, который не знает ни одного языка, но по виду текста — какие буквы и сочетания букв в нём встречаются и как часто — уверенно говорит, русский это или немецкий.

0.3. Общая архитектура обеих систем

Обе системы построены одинаково, и это стоит уметь рассказать как единый принцип:

- **Слой данных** — коллекция документов, эталонная разметка, база или файлы профилей.
- **Ядро** — алгоритмы, не знающие ничего про веб: индексатор и поиск (ЛР 1), методы распознавания и оркестратор (ЛР 2).
- **Веб-интерфейс** — FastAPI + Jinja2: страницы, формы, графики, справка.
- **Сценарии для отчёта** — каталог `tools/`: прогоны метрик, графики, схемы, сборка отчёта из тех же данных.
- **Тесты** — `tests/`: 36 проверок в ЛР 1 и 219 в ЛР 2.
- **Отключаемая надстройка** — паук Пафнутий, достижения, шахматы. Выключается одной переменной окружения (`ARACHNE_COMPANION=0`, `TOLMACH_COMPANION=0`) и никак не влияет на расчёты — это проверяют тесты.

Главная идея — **разделение ответственности**: алгоритм не зависит от интерфейса, интерфейс не вмешивается в измерения, а все параметры собраны в одном месте (`config.py` и `.env`).

Часть 1. ЛР 1 — информационно-поисковая система «Арахна»

1.1. Постановка задачи

Цель работы — освоить на практике основные принципы реализации информационно-поисковых систем и методы оценки качества их работы.

Параметр варианта 3	Значение	Где реализовано
Реализация элементов ИИ	интерфейс с пользователем	arachne.ai/, веб-интерфейс
Сфера применения	локальная вычислительная сеть	crawler.py — обход каталогов и UNC-ресурсов
Стратегия поиска	векторная (TF-IDF + косинусная мера)	indexer.py, search.py
Язык	русский	text/morphology.py — pymorphy3

Требования методички к системе: на входе — множество естественно-языковых текстов; ключевые слова выделяются автоматически по формуле (1.6); пользователь формулирует запрос на естественном языке; на выходе — список релевантных документов с **активной ссылкой** и **списком слов запроса, найденных в документе**; качество оценивается метриками РОМИП программно, из отдельного подменю, таблицами и графиками; интерфейс простой, со справкой.

1.2. Базовые понятия информационного поиска

ОПРЕДЕЛЕНИЕ · информационный поиск

Процесс поиска в большой коллекции, хранящейся, как правило, в памяти компьютеров, некоего **неструктурированного** материала (обычно — документа), удовлетворяющего **информационные потребности**.

Понятие	Что это	Пример из «Арахны»
Информационная потребность	то, что человек на самом деле хочет узнать	как настроить автоматическую раздачу IP-адресов в офисе
Запрос	словесная, всегда неточная запись потребности	«dhcp настройка»
Релевантность	соответствие документа потребности , а не буквально запросу; бывает бинарной или градуированной	в эталоне: 2 — точно отвечает, 1 — частично, 0 — не по теме
Поисковый образ документа (ПОД)	описание документа перечнем ключевых слов с весами (и, возможно, связями, ролями)	вектор весов A_{ij} по леммам документа
Поисковый образ запроса (ПОЗ)	описание запроса в виде, удобном для системы; той же структуры, что ПОД	вектор: 1 для слов запроса, 0,5 для синонимов
Индексирование	построение ПОД всех документов и структур для быстрого поиска	заполнение таблиц terms, postings
Термин, словарь	единица индекса; словарь — упорядоченное множество терминов мощности D	2437 лемм в словаре
Словоформа и лемма	конкретная форма слова и его нормальная (словарная) форма	«сетях», «сетью» → «сеть»

Понятие	Что это	Пример из «Арахны»
Стоп-слова	служебные слова без темы: предлоги, союзы, частицы	221 слово в data/stopwords_ru.txt
Инвертированный индекс	для каждого термина — список документов, где он встречается, с весами	таблица postings
Выдача, сниппет	упорядоченный список результатов; сниппет — фрагмент текста документа	около 300 символов с подсветкой слов запроса

НА ПАЛЬЦАХ · инвертированный индекс

Это предметный указатель в конце учебника: не «на странице 5 есть слова...», а «слово коммутатор — страницы 12, 48, 103». Чтобы ответить на запрос, не нужно читать все документы: достаточно открыть строки указателя для слов запроса.

ПОДВОХ · лемматизация и стемминг

Стемминг отсекает окончания по правилам (алгоритмы Портера, Snowball): «сетями» → «сет». Быстро, но для русского с его чередованиями и супплетивными формами часто ошибается: «шёл» и «идти» стеммер не свяжет. **Лемматизация** находит словарную форму по морфологическому словарю — это и делает `ru morphology3`. Для русского языка разница ощутима: отключение морфологии снизило MAP «Арахны» с 0,634 до 0,564.

1.3. Из чего состоит поисковая система

По методичке информационно-поисковая система решает пять задач: **приём информации и её предварительная обработка; анализ документов и данных** и, возможно, их хранение; **анализ и организация информационных запросов; поиск релевантной информации; выдача запрашиваемой информации**. Для этого она состоит из трёх компонентов:

Компонент	Что делает	В «Арахне»
Агент (паук, кроулер, робот)	собирает информацию о документах, среди которых идёт поиск	<code>crawler.py</code> : обход каталогов и сетевых папок \\сервер\ресурс, инкрементальность, 8 потоков
База данных	хранит всё, что собрали пауки	SQLite, <code>db.py</code> : документы, словарь, инвертированный индекс, журналы, эталон
Поисковый механизм	интерфейс пользователя к базе данных	<code>search.py</code> + веб-интерфейс: ПОЗ, косинус, ранжирование, сниппеты

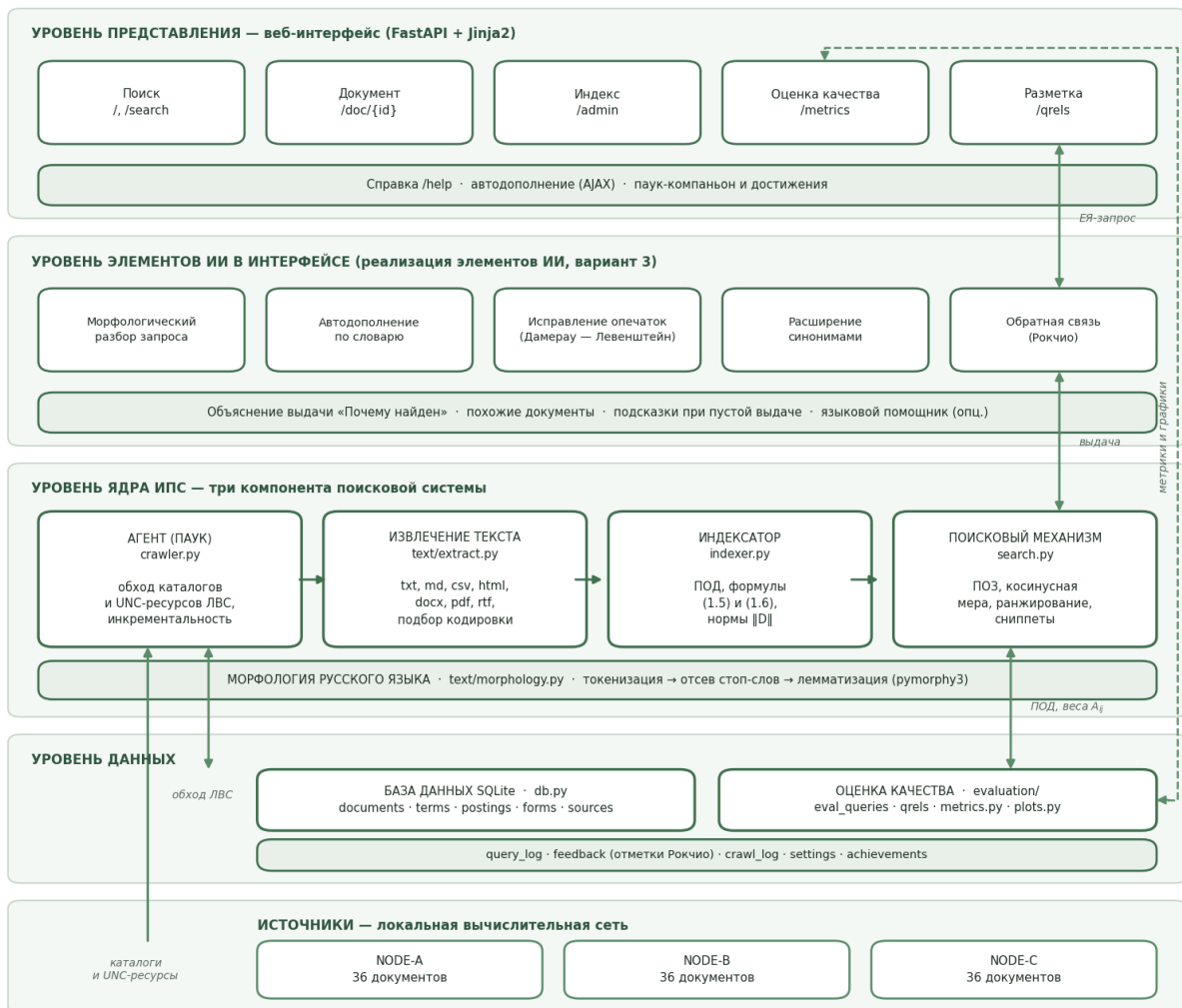


Рисунок 1 — Структура ИПС «Арахна» по уровням (из отчёта по ЛР 1)

1.4. Весовые коэффициенты: TF-IDF

Не все слова одинаково полезны для поиска. Слово, которое есть почти в каждом документе, ничего не говорит о теме; редкое слово — наоборот, «отпечаток пальца» документа. Отсюда два множителя веса.

Обозначение	Название	Смысл
Q_{ij}	частота термина (TF, term frequency)	сколько раз термин i встретился в документе j
P_i	документная частота (DF, document frequency)	в скольких документах базы есть термин i
N	размер коллекции	число документов в базе (у нас 108)
B_i	инверсная частота (IDF)	чем чаще термин по базе, тем меньше
A_{ij}	вес термина в документе (TF-IDF)	итоговая значимость термина i для документа j

$$B_i = \log (N / P_i) \quad (1.5)$$

$$A_{ij} = Q_{ij} \cdot B_i \quad (1.6)$$

В «Арахне» логарифм **десятичный** (LOG_BASE = 10 в indexer.py). Полезные свойства: если термин есть во всех документах, $B_i = \log 1 = 0$ и он не влияет на поиск; максимальное значение $B_i = \log N$ — у термина, встретившегося в одном документе. Логарифм сглаживает шкалу: термин в 10 раз реже получает к весу не «в 10 раз больше», а «+1».

В НАШЕЙ СИСТЕМЕ

Термин «сеть», встречающийся почти в трети из 108 документов, получает $V \approx 0,51$, а термин из единственного документа — $\log_{10}(108 / 1) = 2,033$. Матрица «термин × документ» размером 2437×108 заполнена лишь на **2,17 %**, поэтому в базе хранятся только ненулевые веса — это и есть инвертированный индекс.

Индекс **перестраивается целиком**, а не дособируется: новый документ меняет N , а значит все V_i и все A_{ij} . При частичном обновлении веса оказались бы посчитаны при разных N и ранги стали бы несопоставимы. Полная перестройка 108 документов занимает около 0,3 с.

Методичка отмечает, что современные поисковые машины строят веса богаче: учитывают **положение термина** в документе (заголовок важнее середины), **взаимное расположение** терминов, части речи и морфологию.

1.5. Модели и стратегии поиска

Методичка разделяет **модель описания массива** (векторную) и **модель поиска** (линейную). Словарь — упорядоченное множество терминов мощности D . Документ — двоичный вектор размерности D : 1, если термин входит в документ, иначе 0. Информационный массив L — матрица $N \times D$, строки которой — поисковые образы N документов:

$$L = \{ b_{ij} \}, \quad b_{ij} = 1, \text{ если } t_j \in l_i; \quad b_{ij} = 0 \text{ иначе} \quad (1.7)$$

$$L \cdot q = r \quad (1.8)$$

Здесь q — вектор запроса, r — вектор отклика системы. Умножение матрицы на вектор запроса даёт для каждого документа число совпавших с запросом терминов — простейшую оценку релевантности. Векторная модель поиска заменяет нули и единицы весами TF-IDF, а простой счёт совпадений — косинусом угла.

Три стратегии поиска из таблицы вариантов

	Логическая (булева)	Векторная	Вероятностная
Как ищет	запрос — логическое выражение (И, ИЛИ, НЕ); документ либо подходит, либо нет	документ и запрос — векторы весов; ранг — косинус угла между ними	оценивает вероятность того, что документ релевантен запросу; практическая форма — BM25
Ранжирование	нет, только отбор	есть	есть
Плюсы	высокая точность и предсказуемость: получаешь ровно то, что запросил	частичное совпадение, ранжирование, естественно-языковые запросы	теоретическое обоснование; учитывает длину документа и насыщение частоты; обычно лучшее качество
Минусы	надо заранее знать, что в индексе: слишком мало (низкая полнота) или слишком много (низкая точность)	термины считаются независимыми; бинарный ПОЗ уравнивает все слова запроса	параметры (k_1 , b) надо подбирать
В жизни	библиотечные каталоги, патентный поиск, фильтры интернет-магазинов, gper	классический полнотекстовый поиск, «похожие документы», рекомендации	Elasticsearch, OpenSearch, Apache Lucene и Solr — BM25 по умолчанию

В НАШЕЙ СИСТЕМЕ · логическая стратегия внутри векторной

Методичка называет сведение ЕЯ-запроса к логическому «стратегией с отказами». В «Арахне» это режим **«все слова запроса обязательны»** — логическая конъюнкция поверх векторной модели. Эксперимент показал цену: MAP падает с 0,634 до 0,242, P@10 — с 0,286 до 0,097. Релевантные документы, где тема выражена другими словами, просто отсекаются.

1.6. Векторная модель: TF-IDF и косинусная мера

Каждому документу d ставится в соответствие вектор весов, запросу q — вектор той же размерности:

$$D = \{ w_{d1}, \dots, w_{dn} \}, \quad Q = \{ w_{q1}, \dots, w_{qn} \}, \quad w_{qj} \in \{0; 1\}$$

Мера схожести — косинус угла между векторами:

$$\begin{aligned} \text{sim}(D, Q) &= (D, Q) / (\|D\| \cdot \|Q\|) \\ (D, Q) &= \sum_j w_{dj} \cdot w_{qj}, \quad \|D\| = \sqrt{(\sum_j w_{dj}^2)} \end{aligned} \quad (1.9)$$

НА ПАЛЬЦАХ · почему косинус, а не просто скалярное произведение

Скалярное произведение растёт вместе с длиной документа: длинный текст набирает больше совпадений просто потому, что он длинный. Деление на нормы убирает длину и оставляет **направление** вектора — то есть тему. Для неотрицательных весов косинус лежит в $[0; 1]$: 1 — векторы сонаправлены, 0 — нет ни одного общего термина.

РАЗОБРАННЫЙ ПРИМЕР · считаем ранги вручную

Коллекция из $N = 4$ документов (уже после лемматизации): $D1 = \text{«сеть, коммутатор, сеть»}$; $D2 = \text{«сеть, маршрутизатор»}$; $D3 = \text{«нейронный, сеть, обучение»}$; $D4 = \text{«рецепт, борщ»}$. Запрос «коммутатор в сети» → $PO3 = \{\text{коммутатор: 1, сеть: 1}\}$, $\|Q\| = \sqrt{2} = 1,4142$.

Инверсные частоты: $P(\text{сеть}) = 3 \rightarrow B = \log_{10}(4/3) = 0,1249$; у остальных терминов $P = 1 \rightarrow B = \log_{10} 4 = 0,6021$.

Документ	Веса A_{ij}	$\ D\ $	(D, Q)	sim
D1	сеть $2 \cdot 0,1249 = 0,2499$; коммутатор 0,6021	0,6519	0,8519	0,924
D2	сеть 0,1249; маршрутизатор 0,6021	0,6149	0,1249	0,144
D3	сеть 0,1249; нейронный 0,6021; обучение 0,6021	0,8606	0,1249	0,103
D4	рецепт 0,6021; борщ 0,6021	0,8514	0	0

Вывод. D2 и D3 совпадают с запросом только словом «сеть», но D2 выше: у D3 больше весомых «чужих» терминов, его норма больше, и доля «сети» в нём меньше. Это и есть нормировка на длину. D4 не имеет общих терминов и в выдачу не попадает.

Классы по методичке и их воплощение

Класс методички	Назначение	В коде «Арахны»
Document	документ: заголовок, текст, дата, время, id; добавление и удаление из базы; B_i , A_{ij} , вектор документа	таблица documents, функции indexer.py (get_document_vector_by_lemma и др.)

Класс методички	Назначение	В коде «Арахны»
Search	параметры поиска (все слова вместе, даты, текст запроса); ПОЗ, скалярное произведение, евклидова норма, выдача	датакласс Search в search.py: get_search_query_vector, scalar_product, euclidean_norm, get_search_result
SearchResult	одна строка выдачи: id, заголовок, сниппет, ранг, дата	models.SearchResult + найденные слова, путь, узел, объяснение ранга

1.7. Элементы ИИ в интерфейсе (суть варианта 3)

Вариант 3 требует реализовать элементы искусственного интеллекта именно в **интерфейсе с пользователем**: система должна понимать естественно-языковой запрос, помогать его сформулировать и объяснять свой ответ.

Морфологический разбор запроса

Каждое слово запроса разбирается руторморфиз (словарь OpenCorpora): нормальная форма, часть речи, есть ли слово в словаре системы. Пользователь видит блок «Как система поняла запрос» и отброшенные стоп-слова.

Автодополнение

Подсказки при наборе берутся из двух источников — прежних запросов (query_log) и словоформ индекса (forms) — и упорядочиваются по частоте. Подгружаются асинхронно, без перезагрузки страницы.

Исправление опечаток: расстояние Дамерау — Левенштейна

ОПРЕДЕЛЕНИЕ

Расстояние Левенштейна — минимальное число операций вставки, удаления и замены символа, превращающих одну строку в другую. **Расстояние Дамерау — Левенштейна** добавляет четвёртую операцию — **перестановку двух соседних символов**, самую частую опечатку при быстром наборе.

- «комутатор» → «коммутатор»: 1 (вставка «м»);
- «оепрационная» → «операционная»: 1 (перестановка «еп» → «пе»); у Левенштейна было бы 2 замены;
- порог в «Арахне»: 1 для слов до 5 букв, 2 для более длинных; из кандидатов выбирается ближайший, при равенстве — более частотный. Система предлагает «Возможно, вы имели в виду...».

Расширение запроса синонимами

Тезаурус предметной области data/synonyms_ru.json — 36 групп взаимозаменяемых терминов: коммутатор — свитч, маршрутизатор — роутер, резервный — бэкап, папка — каталог — директория и т. д. Синонимы входят в ПОЗ с **пониженным весом 0,5**, чтобы не перебивать то, что ввёл сам пользователь. Эффект: MAP 0,634 → 0,670, bpref 0,649 → 0,697.

Обратная связь по релевантности: метод Роккио

Пользователь отмечает документы кнопками «Больше таких» и «Не то», и вектор запроса сдвигается:

$$Q' = \alpha \cdot Q + (\beta / |D_r|) \cdot \sum_{d \in D_r} d / \|d\| - (\gamma / |D_{nr}|) \cdot \sum_{d \in D_{nr}} d / \|d\| \quad (1.10)$$

D_r — документы, отмеченные как удачные, D_{nr} — как неудачные. В «Арахне» $\alpha = 1,0$; $\beta = 0,75$; $\gamma = 0,15$; отрицательные компоненты обнуляются, вектор усекается до 40 самых весомых терминов, иначе запрос «расползается» по всей коллекции.

НА ПАЛЬЦАХ

Запрос «подтягивается» к понравившимся документам и немного «отталкивается» от неудачных. γ меньше β , потому что отрицательная отметка менее информативна: документ может быть «не тем» по десятку разных причин, а удачный документ точно указывает направление.

Остальные функции интерфейса

- **Похожие документы** — косинусная мера «документ — документ» по 25 самым весомым терминам образца, выводятся 5 ближайших.
- **Объяснение выдачи «Почему найден»** — вклад каждого термина в ранг: $A_{ij} \cdot w_{qj} / (\|D\| \cdot \|Q\|)$. Сумма вкладов равна рангу документа.
- **Подсказки при пустой выдаче** — снять режим «все слова», проверить исправленный вариант, убрать фильтр по дате, сократить запрос.
- **Языковой помощник (необязательно)** — любой сервис с OpenAI-совместимым API: краткий ответ по найденным документам, другие формулировки запроса. Выключен по умолчанию и не участвует ни в ранжировании, ни в метриках.

1.8. Оценка качества: метрики РОМИП

РОМИП — Российский семинар по оценке методов информационного поиска. Проводится с 2003 года по образцу международных инициатив TREC, CLEF и NTCIR; его цель — независимая оценка методов поиска для русскоязычной информации. Идея оценки проста: люди-ассессоры заранее размечают, какие документы релевантны каким запросам, а система сравнивается с этой разметкой.

Понятие	Что это
Эталонная разметка (qrels)	оценки релевантности «запрос — документ», сделанные экспертом. У нас 36 информационных потребностей и 271 оценка: 52 — «2», 95 — «1», 124 — «0»; в среднем 4,1 релевантного документа на запрос
Пул	документы, которые эксперт реально просмотрел, — обычно верхушка выдачи. Всё, что вне пула, считается неоценённым
Градации	0 — не по теме, 1 — частично полезен, 2 — точно отвечает. Бинарные метрики считают релевантным всё, что > 0
Глубина выдачи top-K	сколько первых документов учитывается; в отчёте $K = 20$

Метрики

Метрика	Формула	Что показывает
Точность P	$P = \text{rel} \cap \text{ret} / \text{ret} $	доля релевантных среди найденных
Полнота R	$R = \text{rel} \cap \text{ret} / \text{rel} $	доля найденных среди всех релевантных
F-мера	$F_1 = 2PR / (P + R)$	гармоническое среднее: наказывает перекос между P и R
P@k	доля релевантных среди первых k	что видит пользователь на первом экране
R-точность	$P@R$, где R — число релевантных для запроса	точность на «честной» глубине для каждого запроса

Метрика	Формула	Что показывает
AP	$AP = (1/R) \cdot \sum_k P@k \cdot rel(k)$	средняя точность в точках, где найден релевантный документ
MAP	среднее AP по всем запросам	главная метрика качества ранжирования
11-точечный график	$P_{int}(r) = \max P(r')$ при $r' \geq r, r = 0; 0,1; \dots; 1$	как падает точность с ростом полноты
DCG	$\sum_i (2^{rel(i)} - 1) / \log_2(i + 1)$	выигрыш с учётом градаций и позиции: чем ниже, тем меньше вклад
nDCG	DCG / IDC (DCG идеального порядка)	DCG, приведённый к [0; 1]
bpref	$(1/R) \cdot \sum_r (1 - n \text{ выше } r / \min(R, N))$	учитывает только оценённые документы; устойчива к неполноте разметки

Обозначения: *rel* — найденные документы, *rel* — релевантные, $rel(k) = 1$, если на позиции *k* релевантный документ; в *bpref* *n* — оценённые нерелевантные документы, стоящие выше релевантного *r*, *N* — число оценённых нерелевантных.

РАЗОБРАННЫЙ ПРИМЕР · все метрики на одной выдаче

Выдача из 10 документов; у запроса всего **R = 6** релевантных, из них 4 найдены, 2 (с оценкой 1) система не нашла. Все нули в выдаче оценены экспертом.

Позиция	1	2	3	4	5	6	7	8	9	10
Оценка	2	0	1	0	0	2	0	0	0	1

P = 4/10 = 0,400; **R** = 4/6 = 0,667; **F₁** = 2 · 0,4 · 0,667 / 1,067 = 0,500.

P@5 = 2/5 = 0,400; **R-точность** = **P@6** = 3/6 = 0,500.

AP = (1/1 + 2/3 + 3/6 + 4/10) / 6 = 2,567 / 6 = **0,428** — делим на все 6 релевантных, а не на 4 найденных: ненайденные документы штрафуют AP.

DCG = 3/log₂2 + 1/log₂4 + 3/log₂7 + 1/log₂11 = 3 + 0,5 + 1,069 + 0,289 = 4,858. Идеальный порядок 2, 2, 1, 1, 1, 1 даёт **IDCG** = 6,567, отсюда **nDCG** = 0,740.

bpref: над релевантными стоят 0, 1, 3 и 6 оценённых нерелевантных; $\min(R, N) = 6$. **bpref** = (1 + (1 - 1/6) + (1 - 3/6) + (1 - 6/6)) / 6 = **0,389**.

Интерполированная точность на уровнях полноты 0; 0,1; ...; 1: 1; 1; 0,667; 0,667; 0,5; 0,5; 0,4; 0; 0; 0; 0.

Все числа проверены функциями `arachne/evaluation/metrics.py`.

Результаты «Арахны»

Метрика	Значение	Как понимать
Точность P (top-20)	0,274	низкая не из-за плохого поиска: релевантных в среднем 4,1, и даже идеальная система на глубине 20 дала бы около 0,2
Полнота R	0,832	система находит подавляющее большинство релевантных
F-мера	0,371	—
P@5 / P@10 / P@20	0,467 / 0,286 / 0,169	точность падает с глубиной — релевантное стоит наверху
R-точность	0,568	больше половины документов на уровне R действительно релевантны
MAP	0,634	основная метрика ранжирования
nDCG / nDCG@10	0,859 / 0,835	выше MAP: документы с оценкой 2 система ставит наверх устойчиво, а 1 от 0 отличает хуже

Метрика	Значение	Как понимать
bpref	0,649	—

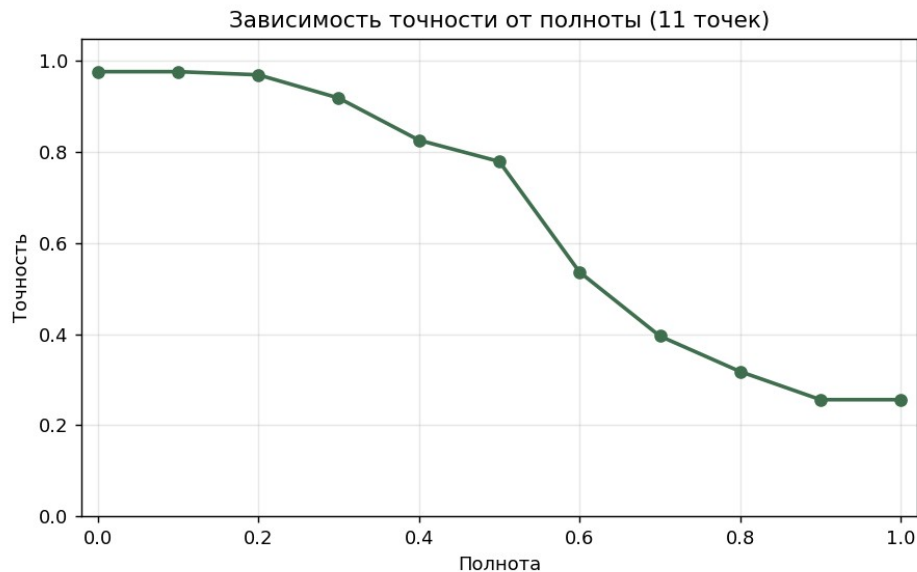


Рисунок 2 — Кривая «точность — полнота»: до полноты 0,5 точность держится выше 0,78, затем резко падает

Сравнение конфигураций

Конфигурация	MAP	P@10	R-точн.	nDCG	bpref
Базовая: леммы, бинарный ПОЗ	0,634	0,286	0,568	0,859	0,649
С расширением синонимами	0,670	0,300	0,596	0,876	0,697
Строгий режим (все слова)	0,242	0,097	0,250	0,475	0,247
Без морфологии (словоформы)	0,564	0,256	0,526	0,808	0,601

Выводы из эксперимента: морфология даёт наибольший вклад (–11 % MAP без неё); синонимы улучшают все метрики умеренно, но устойчиво; строгий режим сводит векторную модель к логической и ухудшает качество более чем вдвое — это подтверждает выбор векторной стратегии.

Худшие запросы — многословные и с межтематической омонимией: «защита от вирусов и программ-вымогателей» (AP 0,312), «алгоритмы сортировки и вычислительная сложность» (0,333), «запросы sql к базе данных» (0,376). Слова «запрос» и «индекс» одинаково частотны в разделах о СУБД и об информационном поиске, а бинарный ПОЗ не позволяет указать, какое слово важнее. Коллекция нарочно построена с такой омонимией («сеть» — ЛВС, нейросеть, железная дорога; «маршрут», «узел», «пакет»), иначе метрики вырождаются.

Как улучшить поиск

- **Логарифмическая частота** $1 + \log Q_{ij}$ вместо Q_{ij} — ослабляет влияние длины документа.
- **Взвешенный ПОЗ:** умножать координаты запроса на B_i , чтобы «sql» весил больше, чем «запрос».
- **Позиция термина:** повышать вес слов из заголовка и первого абзаца.
- **BM25** — вероятностная модель с насыщением частоты и нормировкой на среднюю длину документа:

$$\text{score}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot f(t, D) \cdot (k_1 + 1) / (f(t, D) + k_1 \cdot (1 - b + b \cdot |D| / \text{avgdl}))$$

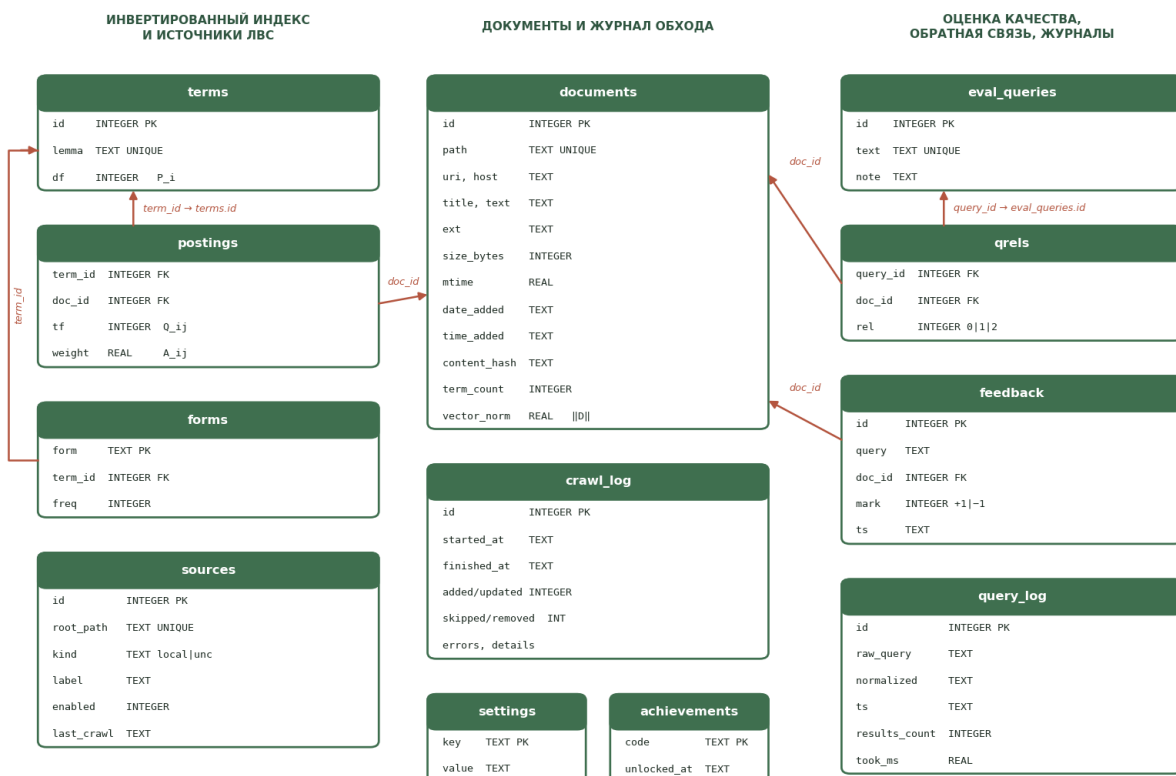
$f(t, D)$ — частота термина в документе, $|D|$ — длина документа, avgdl — средняя длина по коллекции, обычно $k_1 \approx 1,2$ – $2,0$ и $b \approx 0,75$.

- **Двухуровневый тезаурус:** кроме синонимов — отношения «род — вид» («коммутатор» → «сетевое оборудование»).

1.9. Структура приложения «Арахна»

Модуль	Назначение
crawler.py	обход каталогов и UNC-ресурсов ЛВС, инкрементальность по mtime и sha1, журнал обхода
text/extract.py	извлечение текста из txt, md, csv, html, docx, pdf, rtf; подбор кодировки
text/morphology.py	токенизация, стоп-слова, лемматизация (pymorphy3) с кэшем
indexer.py	инвертированный индекс, формулы (1.5) и (1.6), нормы векторов
search.py	поисковый образ запроса, косинусная мера, ранжирование, сниппеты
ai/suggest.py	разбор ЕЯ-запроса, автодополнение, опечатки, синонимы
ai/feedback.py	обратная связь по релевантности (Рокчио), похожие документы
ai/llm.py	необязательный языковой помощник (OpenAI-совместимый сервис)
evaluation/	метрики РОМИП, эталонная разметка, графики, сравнение конфигураций
web/	веб-интерфейс: FastAPI + Jinja2, шаблоны, стили, JavaScript
companion.py	паук Пафнутий и достижения (отключаемо)

База данных



Стрелки — внешние ключи (ON DELETE CASCADE). P_i = terms.df, Q_{ij} = postings.tf, A_{ij} = postings.weight = $Q_{ij} \cdot B_i$, $B_i = \log(N / P_i)$, $||D||$ = documents.vector_norm

Рисунок 3 — Схема базы данных SQLite (стрелки — внешние ключи с каскадным удалением)

Ядро схемы — три таблицы векторной модели: *documents* (документы и нормы их векторов $\|D\|$), *terms* (словарь и P_i в поле *df*), *postings* (инвертированный индекс: Q_{ij} в *tf*, A_{ij} в *weight*). Хранение весов прямо в индексе позволяет искать **одним SQL-запросом**, не открывая тексты; тексты нужны только для сниппетов.

Основные алгоритмы

Обход ЛВС (агент). Для каждого ресурса — рекурсивный обход с пропуском служебных каталогов и неподдерживаемых форматов. Если *mtime* и размер файла совпадают с сохранёнными, файл не читается (**инкрементальность**). Изменившиеся файлы читаются пулом из 8 потоков (задача упирается в ввод-вывод), считается SHA-1; если хеш тот же, обновляется только *mtime*. Ошибки доступа пишутся в журнал, обход не прерывается. Документы исчезнувших файлов удаляются из базы.

ИНДЕКСИРОВАНИЕ

для каждого документа j :

текст $\rightarrow$ токены $\rightarrow$ без стоп-слов $\rightarrow$ леммы

$Q_{ij} \leftarrow$ частота леммы i в документе j

$P_i \leftarrow$ число документов, содержащих лемму i

для каждой леммы i : $B_i = \log_{10}(N / P_i)$ (1.5)

для каждой пары (i, j) : $A_{ij} = Q_{ij} \cdot B_i$ (1.6)

для каждого документа j : $\|D_j\| = \sqrt{\sum_i A_{ij}^2}$

ПОИСК

запрос $\rightarrow$ леммы: $w = 1$; синонимы и исправления: $w = 0,5$

выбрать из *postings* строки с леммами запроса

для каждого документа: $(D, Q) = \sum A_{ij} \cdot w_{qi}$; $\text{rank} = (D, Q) / (\|D\| \cdot \|Q\|)$

фильтры: «все слова обязательны», диапазон дат

сортировка по убыванию *rank*; сниппет с подсветкой

Тестовая коллекция и интерфейс

Коллекция: 108 документов на трёх узлах ЛВС (NODE-A, NODE-B, NODE-C по 36), пять форматов (txt, md, html, docx, pdf), десять тем с намеренно пересекающейся лексикой, 1093 КиБ. Генерируется сценарием *tools/make_collection.py*, поэтому воспроизводима.

Страницы: поиск (*/*, */search*), документ (*/doc/{id}*) и исходный файл (*/raw/{id}*), управление индексом (*/admin*), оценка качества (*/metrics*, */metrics/compare*), разметка релевантности (*/qrels*), достижения, справка (*/help*). Метрики выгружаются в CSV, графики — в PNG.

1.10. Где это применяется в жизни

В ЖИЗНИ · поисковые системы и их потомки

- **Веб-поиск** (Google, Яндекс): инвертированный индекс + ранжирование. TF-IDF и BM25 там — лишь несколько из сотен факторов, но подсказки при наборе, «возможно, вы имели в виду», «похожие страницы» — ровно то, что сделано в «Арахне» в миниатюре.
- **Корпоративный и полнотекстовый поиск**: Elasticsearch, OpenSearch, Apache Solr построены на библиотеке Lucene, где ранжирование по умолчанию — BM25 (с Lucene 6 и Elasticsearch 5.0). Поиск по сетевым папкам, базам знаний, документообороту, e-discovery в юриспруденции.
- **RAG** (генерация, дополненная поиском): языковая модель сначала ищет подходящие фрагменты — часто гибридно, BM25 + векторные эмбединги с **косинусной мерой** — и только потом отвечает. Кнопка «Краткий ответ по найденному» в «Арахне» — это мини-RAG.
- **Рекомендательные системы**: «похожие товары и статьи» — близость векторов; идея Рокчио «сдвинуть запрос к понравившемуся» живёт в лайках и дизлайках лент.
- **Исправление опечаток**: расстояние Дамерау — Левенштейна — в спеллчекерах, поиске по ФИО, дедупликации записей в CRM.
- **Оценка качества**: ассессоры, TREC и POMIP; в индустрии nDCG — стандартная метрика при A/B-тестах ранжирования.

Часть 2. ЛР 2 — распознавание языка текста, система «Толмач»

2.1. Постановка задачи

Цель работы — изучить и отработать практические навыки применения методов автоматического распознавания языка текстовых документов.

Параметр варианта 4	Значение
Языки	русский, немецкий
Формат документов	HTML
Методы	N-грамм, алфавитный, нейросетевой

Требования методички: на входе — документы одинакового размера (около страницы А4) на языках варианта; на выходе — **активная ссылка** на документ, язык каждого текста и **сводная статистика** по тестовой коллекции; средства **сохранения в файл и печати**; простой интерфейс со справкой. Обучающий набор — **от 20 до 120 Кб** на язык. Методы нужно сравнить по точности и быстродействию.

2.2. Основные понятия

ОПРЕДЕЛЕНИЕ · общий принцип распознавания языка

Для каждого языка из конечного набора строится **поисковый образ языка (ПОЯ)**; для входного документа по тем же правилам строится **поисковый образ документа (ПОД)**; выбирается **стратегия сравнения** ПОД и ПОЯ, по которой и выносится решение о языке документа.

Понятие	Что это	В «Толмаче»
Распознавание (идентификация) языка	определение естественного языка, на котором написан текст; англ. language identification, LID	русский или немецкий
ПОЯ	профиль языка, построенный по обучающему корпусу	models/ngram.json, alphabet.json, neural.json
ПОД	профиль входного документа, построенный по тем же правилам , иначе образы несопоставимы	строится при каждом распознавании
Обучающий корпус	тексты с заранее известным языком для построения ПОЯ	46 русских файлов (78,6 Кб) и 6 немецких (41,1 Кб)
Тестовая коллекция	документы, на которых проверяется система; не пересекаются с корпусом	20 HTML: 10 русских, 10 немецких
Эталонная разметка	правильный язык каждого тестового документа	data/labels.csv
Символьная N-грамма	подстрока из N подряд идущих символов; 1 — униграмма, 2 — биграмма, 3 — триграмма	«_до», «дом», «ом_»
Ранг	номер признака в списке, упорядоченном по убыванию частоты	первые 300 N-грамм
Классификатор	алгоритм, относящий объект к одному из классов	каждый из трёх методов + голосование
Точность (accuracy)	доля верно распознанных документов	20 из 20 = 100 %

Понятие	Что это	В «Толмаче»
Матрица ошибок	таблица «эталон × ответ»: на диагонали — верные, вне её — ошибки	для каждого метода 10/0 и 0/10
Быстродействие	время на документ и документов в секунду	алфавитный — 0,39 мс, ≈ 2566 док./с

подвох · две разные «точности»

В ЛР 1 **точность (precision)** — доля релевантных среди найденных. В ЛР 2 **точность (accuracy)** — доля верных ответов классификатора. Слово одно, смысл разный. В ЛР 2 precision тоже есть — по каждому языку отдельно: доля документов, отнесённых к языку, которые действительно на нём написаны.

подвох · объём корпуса — в буквах, а не в байтах

Русский корпус весит 78,6 Кб, немецкий — 41,1 Кб, но букв в них почти поровну: 36 716 и 34 620 (перекося 1,06×). Всё дело в кодировке UTF-8: кириллическая буква занимает **2 байта**, латинская — 1. Сравнивать корпуса нужно по буквам и словам: корпус большего объёма дал бы более подробный профиль и незаслуженное преимущество своему языку.

2.3. Три шага распознавания

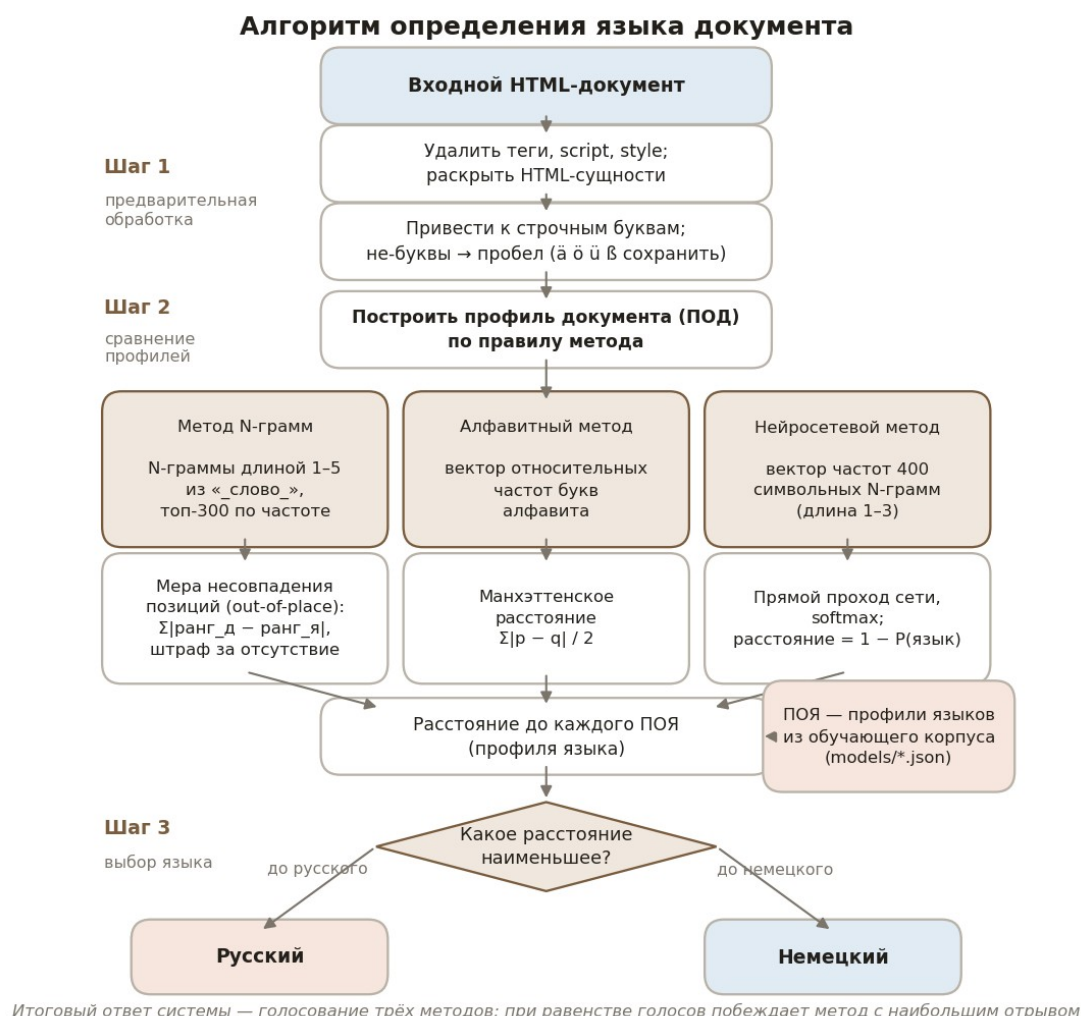


Рисунок 4 — Алгоритм определения языка документа (из отчёта по ЛР 2)

Шаг 1. Предварительная обработка

1. Из HTML извлекается **видимый текст**: удаляются теги и **содержимое** `<script>` и `<style>` (там латиница, которая импортировала бы профиль русского документа), раскрываются HTML-сущности (`&`, ` `, `ä` → `ä`).
2. Текст приводится к **строчным буквам**, всё, что не является буквой, заменяется пробелом. Немецкие `ä ö ü ß` **сохраняются** — без них немецкий профиль теряет заметную часть признаков.
3. Выполняется **нормализация Unicode к форме NFC**.

НА ПАЛЬЦАХ · зачем нормализация NFC

В Unicode букву «ü» можно записать одним символом (U+00FC) или двумя: «u» + комбинируемый знак диерезиса (U+0308). На экране они неотличимы, но для программы это разные строки, и N-граммы получились бы разными. Форма **NFC** (composed) сводит обе записи к одному символу.

Шаг 2. Сравнение профилей

Для каждого языка заранее построен ПОЯ; по тем же правилам строится ПОД; метод считает расстояние от ПОД до каждого ПОЯ. Во всех трёх методах расстояние приведено к отрезку [0; 1], иначе их нельзя было бы сопоставлять.

Построение поисковых образов и их сравнение



Профили языков строятся один раз и сохраняются в `models/*.json`; профиль документа — при каждом распознавании

Рисунок 5 — Оба образа строятся по одному правилу: ПОЯ — один раз из корпуса, ПОД — при каждом распознавании

Шаг 3. Выбор языка

Побеждает язык с **наименьшим расстоянием**. Итоговый ответ системы — **голосование трёх методов** (раздел 2.8).

2.4. Обзор методов распознавания

Методичка перечисляет методы, использующие знания о языке **от уровня алфавита до лексико-грамматического уровня**: коротких слов, частотных слов, N-грамм, алфавитный, грамматических слов, неграмматических слов. Все они достаточно дешёвы для промышленных систем обработки текста. Нейросетевой метод стоит особняком: решающее правило в нём не задаётся вручную, а выводится из данных.

Метод	ПОЯ	Сравнение	Сильная сторона	Слабая сторона
Алфавитный	частоты букв алфавита	расстояние между распределениями частот	самый быстрый; идеален при разных системах письма	бессилен при общем алфавите (немецкий — французский)
N-грамм (Кавнар — Тренкл)	~300 самых частых N-грамм с рангами	мера несовпадения позиций (out-of-place)	устойчив к опечаткам и теме, работает на коротких текстах	медленнее; нужен профиль приличного объёма
Частотных слов	самые частые слова корпуса и их вероятности	пословное сравнение с учётом вероятностей	точен на длинных текстах	в коротком тексте мало слов
Коротких слов	слова не длиннее порога (например, ≤ 5 букв, встретившиеся > 3 раз) и их вероятности	вероятность предложения = произведение вероятностей лексем; отсутствующим — минимальная	короткие слова — в основном служебные, характерны для языка и не зависят от темы	нужно разбиение на слова; языки со сходной служебной лексикой
Грамматических / неграмматических слов	служебные (предлоги, союзы, артикли) либо знаменательные слова языка	аналогично методам слов	грамматические слова — устойчивый признак языка, не зависящий от темы	нужен надёжный список слов; в методичке методы только названы
Нейросетевой	обученная сеть над частотами признаков	вероятность языка на выходе сети	граница решения выучивается из данных	нужно обучение и хранение весов

НА ПАЛЬЦАХ · методы слов — это наивный Байес

Метод коротких слов по сути — **наивный байесовский классификатор**: вероятность текста считается произведением вероятностей его лексем в предположении, что они независимы. На практике перемножают не сами вероятности (произведение сотен малых чисел уходит в машинный ноль), а складывают их логарифмы.

2.5. Метод N-грамм (Кавнар — Тренкл, 1994)

Метод предложили **W. B. Cavnar** и **J. M. Trenkle** в 1994 году (статья «N-Gram-Based Text Categorization», реализация известна как TextCat) для определения языка и кодировки документа по статистике документов с заранее известным языком. В методичке фамилия записана как «Канвар» — правильно **Кавнар**.

Закон Зипфа

ОПРЕДЕЛЕНИЕ

Закон Зипфа (Ципфа) — эмпирическая закономерность: если упорядочить слова языка (или достаточно длинного текста) по убыванию частоты, частота слова окажется примерно обратно пропорциональной его рангу r .

$$f(r) \approx C / r$$

Первое по частоте слово встречается примерно вдвое чаще второго и втрое чаще третьего. Отсюда вывод для метода: небольшое число самых частых элементов покрывает огромную долю текста и определяется **языком**, а не темой. Около 300 самых частых N-грамм — устойчивая «подпись» языка.

Построение профиля

1. Текст разбивается на слова; каждое слово обрамляется заполнителем: «_слово_».

- Из каждой обрамлённой строки извлекаются **все подстроки длиной от 1 до N**; в методичке и в «Толмаче» **N = 5**.
- N-граммы подсчитываются и сортируются по убыванию частоты (при равной частоте — по самой N-грамме, чтобы профиль не зависел от порядка обхода словаря).
- Первые **300** N-грамм образуют профиль; номер N-граммы в списке — её **ранг**.

РАЗОБРАННЫЙ ПРИМЕР · N-граммы слова «дом»

Обрамляем: «_дом_» (длина 5). Длина 1: _, д, о, м, _ · длина 2: _д, до, ом, м_ · длина 3: _до, дом, ом_ · длина 4: _дом, дом_ · длина 5: _дом_.

Итого **15** N-грамм, из них 14 разных («_» встречается дважды). Заполнитель делает значимой **позицию** буквы: «_де» — начало слова, «де_» — его конец, и это разные признаки.

Мера несовпадения позиций (out-of-place)

Для каждой N-граммы профиля документа берётся модуль разности её рангов в профиле документа и профиле языка. N-грамме, которой в профиле языка нет, назначается **максимальный штраф**, равный размеру профиля. В «Толмаче» сумма ещё делится на (число N-грамм документа × штраф), чтобы расстояние лежало в [0; 1]:

$$d(\text{ПОД}, \text{ПОЯ}) = \sum_{g \in \text{ПОД}} \Delta(g) / (|\text{ПОД}| \cdot \text{штраф}), \quad \Delta(g) = |r_{\text{док}}(g) - r_{\text{яз}}(g)| \text{ или штраф}$$

РАЗОБРАННЫЙ ПРИМЕР · out-of-place на игрушечных профилях

Профиль языка (ранги 0–4): «_», «е», «т», «_т», «ст». Профиль документа (ранги 0–3): «_», «т», «е», «qz». Штраф = размер профиля = 5.

N-грамма	ранг в документе	ранг в языке	вклад
«_»	0	0	0
«т»	1	2	1
«е»	2	1	1
«qz»	3	нет	5 (штраф)

Сумма = 7; нормированное расстояние = $7 / (4 \cdot 5) = 0,35$. Чем больше общих N-грамм и чем ближе их ранги, тем меньше расстояние.

2.6. Алфавитный метод

Самый мелкий уровень знаний о языке: язык описывается не словами и не сочетаниями букв, а **распределением частот отдельных букв**. Профиль — вектор относительных частот $p(\text{буква}) = \text{число вхождений} / \text{число всех букв}$. Буква считается частью алфавита языка, если её доля в корпусе выше порога 10^{-4} : так из алфавита выпадают случайные латинские вкрапления в русском тексте и наоборот.

Расстояние — **манхэттенское**, посчитанное по объединению алфавитов и делённое на 2 (теоретический максимум для двух распределений), — так называемое расстояние полной вариации:

$$d(p, q) = \frac{1}{2} \cdot \sum_k |p_k - q_k|$$

Буква, которой в языке нет, отдаёт в сумму всю свою частоту, поэтому для языков с **разными системами письма** расстояние до «чужого» языка сразу близко к максимуму.

РАЗОБРАННЫЙ ПРИМЕР

Языки: RU = {а: 0,5; б: 0,3; в: 0,2}, DE = {х: 0,6; у: 0,4}. Документ: {а: 0,4; б: 0,4; х: 0,2}.

До RU: $\frac{1}{2} \cdot (|0,4 - 0,5| + |0,4 - 0,3| + |0 - 0,2| + |0,2 - 0|) = \frac{1}{2} \cdot 0,6 = \mathbf{0,30}$.

До DE: $\frac{1}{2} \cdot (0,4 + 0,4 + |0,2 - 0,6| + 0,4) = \frac{1}{2} \cdot 1,6 = \mathbf{0,80}$. Документ отнесён к русскому.

подвох · граница алфавитного подхода

Слово «числа», записанное латиницей как **chisla**, все три метода «Толмача» относят к немецкому: профили русского языка построены по кириллице, и латинская запись для них выглядит немецкой. Человек в игре с Пафнутием распознаёт такие слова лучше системы. Проверяется тестом `test_system_fails_on_transliterated_russian`.

2.7. Нейросетевой метод

Признаки

Документ описывается вектором **относительных частот 400 символьных N-грамм длиной 1–3**. Пробел между словами не выбрасывается: N-граммы «де» и «де » говорят, какими буквами язык начинает и заканчивает слова. Словарь признаков отбирается **поровну от каждого языка** («по кругу»), иначе более объёмный корпус вытеснил бы признаки второго языка.

Архитектура: многослойный персептрон

Вход — 400 признаков, **скрытый слой — 32 нейрона с гиперболическим тангенсом**, выход — 2 нейрона с функцией **softmax**. Сеть написана на numpy без готовых фреймворков.

$$h = \tanh(x \cdot W_1 + b_1)$$

$$z = h \cdot W_2 + b_2, \quad P_i = \exp(z_i) / \sum_j \exp(z_j)$$

Softmax превращает произвольные числа (логиты) в вероятности: все P_i положительны и в сумме дают 1. Для устойчивости из логитов вычитается максимум — иначе экспонента переполняется.

Обучение

- **Функция потерь** — перекрёстная энтропия $L = -\sum_i y_i \cdot \log P_i$ (y — правильный ответ в one-hot виде) плюс **L2-регуляризация** с коэффициентом 10^{-4} , штрафующая слишком большие веса.
- **Обратное распространение ошибки (backpropagation)**: градиенты потерь по всем весам считаются по цепному правилу от выхода к входу. Для пары softmax + перекрёстная энтропия градиент по логитам особенно прост: $P - y$. Через tanh градиент умножается на $(1 - h^2)$.
- **Оптимизатор Adam**: для каждого веса хранятся скользящие средние градиента и его квадрата ($\beta_1 = 0,9$; $\beta_2 = 0,999$; $\epsilon = 10^{-8}$) с поправкой смещения; шаг обучения 0,01. Каждый вес получает свой адаптивный шаг.
- **Мини-батчи** по 32 примера, 220 эпох, зерно генератора случайных чисел 17 — результаты воспроизводимы.
- **Контрольная выборка** — 20 % корпуса, отделяется **по тексту до нарезки на фрагменты**. Сохраняются веса лучшей по контрольной выборке эпохи, а не последней.
- **Обучение на нескольких масштабах**: фрагменты 150, 300 и 600 символов с перекрытием 65 %.

НА ПАЛЬЦАХ · зачем несколько длин фрагментов

Короткий текст даёт **разреженный** вектор частот: большинство из 400 N-грамм в нём просто не встретились. Сеть, обученная только на длинных фрагментах, такого входа «не видела» — и точность на коротком тексте проседала до 90 %. Обучение сразу на 150, 300 и 600 символах сняло проблему: на всех длинах от 10 до 1200 символов — 100 %.

Расстояние

Чтобы правило «побеждает наименьшее расстояние» работало одинаково во всех трёх методах, расстоянием считается **$1 - P(\text{язык})$** .

РАЗОБРАННЫЙ ПРИМЕР · softmax и расстояние

Логиты сети: $z = (2,0; 0,5)$. $e^{2,0} = 7,389$, $e^{0,5} = 1,649$, сумма 9,038.

$P(\text{ru}) = 7,389 / 9,038 = \mathbf{0,818}$, $P(\text{de}) = \mathbf{0,182}$. Расстояния: $1 - P = (0,182; 0,818) \rightarrow$ русский.

Обучение занимает около 4 секунд, веса занимают 243 Кб. Главное отличие от первых двух методов: **граница решения не задана формулой вручную, а выведена из данных**.

В ЖИЗНИ · тот же принцип в браузере

Google **CLD3** — нейросетевой детектор языка, рассчитанный на работу в браузере Chrome, — устроен очень похоже: из текста извлекаются символьные N-граммы, считаются их доли, они превращаются в плотные векторы (эмбединги), проходят скрытый слой с ReLU и softmax. Нейросетевой метод «Толмача» — та же схема в миниатюре, только вместо эмбедингов — сами частоты, вместо ReLU — tanh.

2.8. Голосование методов и уверенность

Итоговый ответ — **большинство голосов** трёх методов. При равенстве голосов побеждает язык, за который высказался метод с наибольшей **уверенностью** — относительным отрывом ближайшего языка от второго:

$$\text{уверенность} = (d_2 - d_1) / d_2, \quad d_1 \leq d_2 \text{ — два наименьших расстояния}$$

Для примера с softmax выше: $(0,818 - 0,182) / 0,818 = 0,777$. Ноль означает, что расстояния совпали и выбор случаен. **Цена голосования** — время: чтобы проголосовать, нужно выполнить все методы, поэтому 13,4 мс на документ против 0,39 мс у одного алфавитного.

2.9. Оценка и эксперименты**Тестовая коллекция**

- 20 HTML-документов, 10 русских и 10 немецких, примерно по странице A4 (2155–2891 букв).
- **Темы попарно совпадают** между языками: маяки и Leuchttürme, пчёлы и Imkereei, хлеб и Brot, карты и Kartografie. Иначе методы могли бы различать документы по теме, а не по языку, и оценка оказалась бы завышенной.
- **Разметка неоднородна**: разные шаблоны, `<style>` и `<script>` с латиницей, меню и подвалы — проверка шага 1. Два русских документа содержат настоящие латинские вкрапления (1–2 % букв).
- Темы коллекции **не встречаются в обучающем корпусе**: проверяется обобщение, а не запоминание.

Точность и быстродействие

Метод	Точность	Верно	мс / док.	док. / с
Метод N-грамм	100 %	20 / 20	8,28	≈ 121
Алфавитный	100 %	20 / 20	0,39	≈ 2566
Нейросетевой	100 %	20 / 20	4,76	≈ 210
Согласованное решение (голосование)	100 %	20 / 20	13,43	≈ 74

Все методы безошибочны, разногласий нет ни по одному документу. Это **свойство пары языков**, а не заслуга реализации: кириллица и латиница не пересекаются, задача решается уже на уровне алфавита.

Опыт 1. Зависимость от длины входа

Каждый документ нарезался на окна по границам слов длиной 10, 20, 40, 80, 150, 300, 600 и 1200 символов (48–100 фрагментов на длину). Все три метода — 100 % на всех длинах: для этой пары языков хватает одного-двух слов. Опыт всё равно был полезен — именно он выявил провал нейросети на коротком входе при обучении на одной длине.

Опыт 2. Смешанный текст

Русский документ постепенно замещался немецким. **Точка перелома** — доля немецкого, при которой метод начинает отвечать «немецкий»:

Метод	40 %	45 %	47,5 %	50 %	52,5 %	55 %	Перелом
N-грамм	0	60	90	100	100	100	45 %
Алфавитный	0	0	0	20	100	100	52,5 %
Нейросетевой	0	0	0	50	100	100	50 %

В ячейках — процент смесей, отнесённых к немецкому.

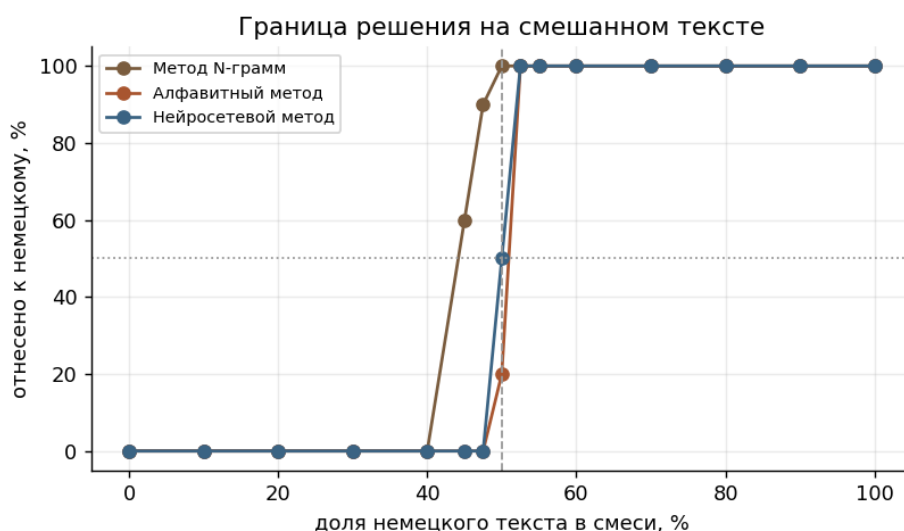


Рисунок 6 — Граница решения на смешанном тексте

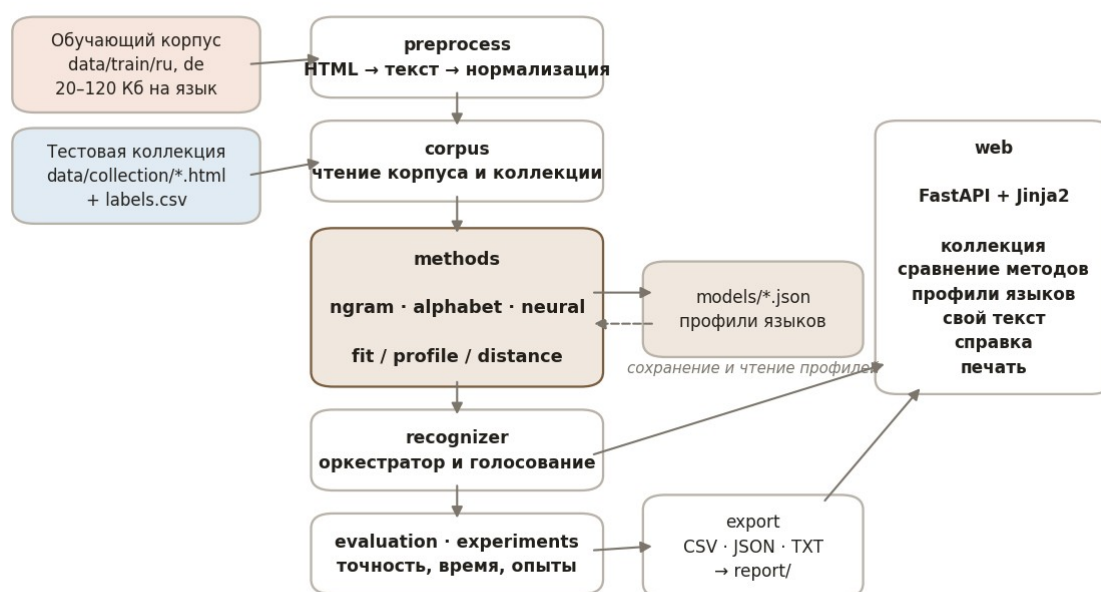
Почему так: метод N-грамм склоняется к немецкому раньше середины — немецкие слова длиннее, и при той же длине текста их N-грамм в профиль приходит больше. Алфавитный дольше держится за русский. Нейросеть делит равную смесь ровно пополам: обучение выровняло её границу, тогда как у двух других граница задана видом метрики и смещена свойствами языков.

Выводы

1. Для языков с **разными системами письма** достаточно алфавитного метода: он так же точен и в 20 раз быстрее метода N-грамм.
2. Ценность методов N-грамм и нейросетевого раскрывается на парах с **общим алфавитом** (немецкий — французский, испанский — итальянский), где решение принимается по оттенкам частот. Алфавитный метод там опустил бы почти до угадывания.
3. Голосование не улучшило точность — улучшать было нечего, — но стоит суммы времён всех методов. Оно оправдано, когда методы **ошибаются по-разному**.
4. Нейросетевой метод требует обучения и хранения весов, зато его граница решения выведена из данных.

2.10. Структура приложения «Толмач»

Структура системы «Толмач»



Методы подключаются через общий интерфейс Method: новый метод добавляется правкой одного реестра

Рисунок 7 — Структура системы «Толмач»

Модуль	Назначение
config.py	пути, языки, параметры методов, выключатель игры
preprocess.py	шаг 1: HTML → текст → нормализация
corpus.py	чтение обучающего корпуса и тестовой коллекции
methods/base.py	общий интерфейс методов и сохранение профилей
methods/ngam.py, alphabet.py, neural.py	три метода: правило построения образа + метрика
recognizer.py	оркестратор: шаги 2 и 3, голосование
evaluation.py, experiments.py	точность, матрицы ошибок, быстродействие; опыты с длиной и смесью
export.py	сохранение результатов в CSV, JSON, TXT
web/app.py	страницы на FastAPI + Jinja2

Модуль	Назначение
chess/, quiz.py, pafnuty.py, web/game.py	игровая надстройка: шахматы с Пафнутием, викторина «угадай язык слова»

НА ПАЛЬЦАХ · общий интерфейс методов — паттерн «Стратегия»

Все методы реализуют одинаковые операции: **обучиться** на корпусе, **построить профиль** текста, **посчитать расстояние** между профилями. Оркестратор не знает, как устроен конкретный метод, — он просто вызывает эти операции у каждого. Новый метод добавляется правкой одного реестра. Это классический паттерн проектирования «Стратегия».

Структуры данных

Структура	Назначение
Document	входной документ после шага 1: текст, число букв и слов, эталонный язык
Profile	поисковый образ: признак → ранг и признак → частота; одна структура и для ПОЯ, и для ПОД
MethodResult	решение одного метода: расстояния до языков, выбранный язык, время, уверенность
Verdict	сводное решение по документу: результаты всех методов, итоговый язык, согласие
MethodScore	качество метода: точность, матрица ошибок, точность и полнота по языкам, время
Report	результат прогона всей коллекции

Страницы: коллекция и сводка (/), разбор документа (/document/<файл>), исходный HTML (/raw/<файл>), сравнение методов (/compare), профили языков (/profiles), свой текст или файл (/check), справка, версия для печати (/print). **Сохранение:** /export/csv, /export/json, /export/txt. **Печать:** правила @media print убирают навигацию и кнопки и не дают таблицам разрываться посередине строки.

Грабли, на которые наступили: инженерные уроки

Проблема	Что произошло	Как решено
Утечка данных	если делить корпус на обучение и контроль после нарезки, перекрывающиеся фрагменты попадают в обе части — контрольная точность завышена	деление по тексту до нарезки
Кэш по id (объекта)	адреса освобождённых объектов в Python переиспользуются — метод возвращал ответ для предыдущего документа; в опыте это выглядело как ровно 50 %	в кэше хранится сам профиль; тест test_alternating_documents_d o_not_leak
Невоспроизводимость	при равных частотах порядок признаков зависел от обхода словаря	зерно 17 + сортировка по самому признаку
Обход каталогов	имя файла в /raw/<имя> могло выйти за пределы коллекции (./.)	проверка пути
Ответы в браузере	решение шахматной задачи и язык слова викторины читались бы в коде страницы	проверка только на сервере

2.11. Где это применяется в жизни

В жизни · распознавание языка и N-граммы вокруг нас

- **Браузеры** предлагают перевести страницу, когда её язык отличается от вашего; Google CLD3 рассчитан именно на работу в Chrome.
- **Машинный перевод** — поле «Определить язык» в переводчиках.
- **Поисковые системы**: язык страницы определяет, какую морфологию и стоп-слова применять при индексации. Связь с ЛР 1 прямая: ru morphology3 годится только для русского, и перед индексацией нужно знать язык документа.
- **Подготовка данных для больших языковых моделей**: веб-корпуса фильтруют по языку; модель fastText lid.176 распознаёт 176 языков и используется, например, в конвейере NVIDIA NeMo Curator.
- **Модерация и поддержка**: маршрутизация обращений к оператору, владеющему языком клиента; аналитика соцсетей по языкам.
- **N-граммы шире**: автодополнение и T9, проверка орфографии, антиплагиат (шинглы — перекрывающиеся фрагменты текста), k-меры в биоинформатике для сравнения последовательностей ДНК, сигнатуры вредоносного кода.
- **Готовые библиотеки**: langdetect (наивный Байес над символьными N-граммами), langid.py, fastText, CLD3, Lingua. В «Толмаче» ни одна не используется — все три метода, включая нейросеть, написаны с нуля.

Часть 3. Технологии и компоненты

Обе системы собраны из одних и тех же кирпичей. Для защиты полезно знать не только «что это», но и **почему выбрано** и **какие есть особенности применения**, — этого требуют обе методички в разделе «готовые компоненты».

Компонент	Где	Что делает	Особенности применения	Где встречается в жизни
Python 3.14	ЛР 1, 2	язык реализации	богатая экосистема для обработки текста и данных	наука о данных, ИИ, бэкенд
FastAPI + Uvicorn	ЛР 1, 2	веб-каркас (маршруты, формы) и ASGI-сервер	синхронные обработчики выполняются в пуле потоков, поэтому каждому запросу — своё подключение к SQLite	API сервисов, выдача ML-моделей в продакшене
Jinja2	ЛР 1, 2	шаблоны HTML-страниц на сервере	графики рисуются прямо в шаблоне как inline-SVG — нет зависимости от внешних CDN	Flask, Ansible, генераторы сайтов
SQLite	ЛР 1	встроенная СУБД в одном файле	режим WAL, внешние ключи с каскадным удалением, пакетные вставки	браузеры, Android и iOS, настольные приложения
pymorphy3	ЛР 1	морфологический анализ русского языка по словарю OpenCorpora	несколько вариантов разбора — берётся самый вероятный; кэш <code>lgu_cache</code> , иначе индексация замедляется в разы	NLP для русского языка
BeautifulSoup4	ЛР 1, 2	разбор HTML	удаляются <code>script</code> и <code>style</code> ; в ЛР 2 есть запасной разборщик на <code>html.parser</code>	веб-скрейпинг, очистка данных
python-docx, rypdf	ЛР 1	извлечение текста из DOCX и PDF	читаются и абзацы, и таблицы; ошибка на одной странице PDF не прерывает разбор	документооборот, поиск по файлам
numpy	ЛР 2	матричные операции нейросети	единственная внешняя библиотека в расчётах ЛР 2	основа научного Python: pandas, scikit-learn
matplotlib	ЛР 1, 2 (tools/)	графики и схемы для отчёта	<code>pdf.fonttype = 42</code> , иначе кириллица из PDF извлекается неверно	научная графика
pytest + httpx	ЛР 1, 2	автотесты, в том числе веб-страниц	36 тестов в ЛР 1, 219 — в ЛР 2; отдельный набор проверяет режим без надстройки	непрерывная интеграция
LLM по OpenAI-совместимому API	ЛР 1 (опц.)	языковой помощник над выдачей	вызов через <code>urllib</code> без SDK; выключен по умолчанию; 403 от сервиса лечится сменой провайдера, а не VPN	чат-боты, RAG-системы
JSON, CSV	ЛР 1, 2	хранение профилей, эталона, выгрузок	эталон сопоставляется по имени файла, а не по <code>id</code> , — переживает пересборку базы	обмен данными

3.1. Сквозные архитектурные принципы

- **Разделение слоёв:** данные — алгоритмы — интерфейс. Алгоритм можно запустить из консоли, из теста и из веба одинаково.
- **Единый интерфейс взаимозаменяемых частей:** три метода распознавания с одинаковыми операциями; конфигурации поиска, сравниваемые одним модулем.

- **Воспроизводимость:** коллекции генерируются сценариями, зерно генератора зафиксировано, признаки упорядочиваются детерминированно, отчёт собирается из тех же данных, что и интерфейс.
- **Измерения неприкосновенны:** игровая надстройка отключается одной переменной окружения и не касается метрик — это проверено тестами.
- **Конфигурация в одном месте:** `config.py` и `.env`.
- **Защитное программирование:** проверка путей в `/raw`, ответы игр проверяются только на сервере, сбой внешнего сервиса не ломает основную функцию.

Часть 4. Шпаргалка формул

Формула	Обозначения	Где
$B_i = \log(N / P_i)$	N — документов в базе, P_i — документов с термином i	ЛР 1, (1.5)
$A_{ij} = Q_{ij} \cdot B_i$	Q_{ij} — частота термина i в документе j	ЛР 1, (1.6)
$L \cdot q = r$	L — матрица $N \times D$, q — вектор запроса, r — отклик	ЛР 1, (1.8)
$\text{sim}(D, Q) = (D, Q) / (\ D\ \cdot \ Q\)$	скалярное произведение и евклидовы нормы	ЛР 1, косинус
$Q' = \alpha Q + \beta / D_r \cdot \sum d / \ d\ - \gamma / D_{nr} \cdot \sum d / \ d\ $	$\alpha = 1$; $\beta = 0,75$; $\gamma = 0,15$	ЛР 1, Рокчио
$P = \text{rel} \cap \text{ret} / \text{ret} $; $R = \text{rel} \cap \text{ret} / \text{rel} $	ret — найденные, rel — релевантные	ЛР 1
$F_1 = 2PR / (P + R)$	гармоническое среднее	ЛР 1
$AP = (1/R) \cdot \sum_k P@k \cdot \text{rel}(k)$; MAP = среднее AP	R — все релевантные запроса	ЛР 1
$P_{\text{int}}(r) = \max P(r')$, $r' \geq r$	11 уровней полноты 0; 0,1; ...; 1	ЛР 1
$DCG = \sum (2^{\text{rel}} - 1) / \log_2(i + 1)$; $nDCG = DCG / \text{IDCG}$	i — позиция, rel — градация 0/1/2	ЛР 1
$\text{bpref} = (1/R) \cdot \sum_r (1 - n \text{ выше } r / \min(R, N))$	n — оценённые нерелевантные, N — их число	ЛР 1
$\text{score} = \sum \text{IDF} \cdot f(k_i + 1) / (f + k_i(1 - b + b D /\text{avgdl}))$	BM25; $k_1 \approx 1,2-2$, $b \approx 0,75$	улучшение ЛР 1
$f(r) \approx C / r$	закон Зипфа: частота обратно пропорциональна рангу	ЛР 2
$d = \sum r_{\text{док}}(g) - r_{\text{яз}}(g) / (\text{ПОД} \cdot \text{штраф})$	out-of-place; отсутствующей N-грамме — штраф = размер профиля	ЛР 2, N-граммы
$d(p, q) = \frac{1}{2} \cdot \sum_k p_k - q_k $	p, q — частоты букв	ЛР 2, алфавитный
$h = \tanh(xW_1 + b_1)$; $P = \text{softmax}(hW_2 + b_2)$	$400 \rightarrow 32 \rightarrow 2$	ЛР 2, нейросеть
$\text{softmax}: P_i = \exp(z_i) / \sum_j \exp(z_j)$	z — логиты	ЛР 2
$L = -\sum y_i \log P_i + \lambda \ W\ ^2$	перекрёстная энтропия + L2, $\lambda = 10^{-4}$	ЛР 2
$d = 1 - P(\text{язык})$; уверенность = $(d_2 - d_1) / d_2$	$d_1 \leq d_2$	ЛР 2
accurasy = верных / всего	доля правильных ответов классификатора	ЛР 2

Часть 5. Вопросы для самопроверки

Закройте ответ и попробуйте ответить сами. Вопросы подобраны так, как их обычно задают на защите: от определений к «почему именно так».

5.1. По ЛР 1

1. Из каких трёх компонентов состоит поисковая система?

Агент (паук, кроулер) собирает документы; база данных хранит собранное и индекс; поисковый механизм — интерфейс пользователя к базе.

2. Чем ПОД отличается от ПОЗ и что у них общего?

ПОД описывает документ, ПОЗ — запрос. Внутри одной системы у них обязательно одинаковая структура, иначе их нельзя сравнить. В «Арахне» оба — векторы по леммам.

3. Что такое инверсная частота термина и зачем логарифм?

$B_i = \log(N / P_i)$ — чем в большем числе документов встречается термин, тем меньше его вес. Логарифм сглаживает шкалу: в 10 раз более редкий термин получает +1, а не $\times 10$. У термина, который есть во всех документах, $B = 0$.

4. Почему индекс перестраивается целиком?

Новый документ меняет N , а значит все B_i и A_{ij} . Частичное обновление дало бы веса, посчитанные при разных N , — ранги стали бы несопоставимы.

5. Зачем делить скалярное произведение на нормы?

Чтобы убрать влияние длины документа: остаётся направление вектора, то есть тема. Иначе длинные документы всегда были бы наверху.

6. Чем векторная стратегия лучше логической для ЕЯ-запросов?

Допускает частичное совпадение и ранжирует. Логическая требует точно знать, что в индексе; эксперимент: режим «все слова» уронил MAP с 0,634 до 0,242.

7. Зачем лемматизация и чем она лучше стемминга для русского?

Сводит «сети», «сетей», «сетью» к одному термину. Стеммер отсекает окончания по правилам и путается в чередованиях; лемматизатор берёт словарную форму. Без морфологии MAP падает на 11 %.

8. Что такое расстояние Дамерау — Левенштейна?

Минимальное число вставок, удалений, замен и перестановок соседних символов. «оопрационная» → «операционная» — 1 (перестановка), по Левенштейну было бы 2.

9. Как работает метод Рокчио и почему $\gamma < \beta$?

Вектор запроса сдвигается к средним векторам удачных документов и от неудачных. Отрицательная отметка менее информативна: «не то» бывает по многим причинам.

10. Почему синонимы входят в запрос с весом 0,5?

Чтобы расширение не перебивало слова, которые ввёл сам пользователь, — синоним менее надёжен, чем исходное слово.

11. Чем точность отличается от полноты? Почему у «Арахны» точность 0,27 — это не плохо?

Точность — доля релевантных среди найденных, полнота — доля найденных среди релевантных. На глубине 20 при 4,1 релевантного в среднем даже идеальная система дала бы около 0,2.

12. Что такое AP и MAP?

AP — среднее значений точности в позициях, где найден релевантный документ, делённое на число всех релевантных. MAP — среднее AP по запросам; главная метрика ранжирования.

13. Чем nDCG отличается от MAP?

nDCG учитывает градации релевантности (2 важнее 1) и дисконтирует вклад по позиции логарифмом. У «Арахны» nDCG 0,859 > MAP 0,634: документы с оценкой 2 стоят наверху устойчиво.

14. Зачем нужна bpref?

Эталон всегда неполон. bpref учитывает только оценённые документы и считает, сколько оценённых нерелевантных стоит выше каждого релевантного, — неоценённые документы её не портят.

15. Что показывает 11-точечный график и как строится?

Как падает точность с ростом полноты. Для уровней полноты 0; 0,1; ...; 1 берётся интерполированная точность — максимум точности при полноте не ниже данной, — и усредняется по запросам.

16. Что такое РОМИП?

Российский семинар по оценке методов информационного поиска, с 2003 года, по образцу TREC, CLEF, NTCIR. Методичка ссылается на его набор метрик.

17. Как бы вы улучшили поиск?

Логарифмическая TF, взвешенный ПОЗ через B_i , бонус за заголовок, BM25, тезаурус с отношением «род — вид».

18. Что такое инкрементальный обход?

Паук не перечитывает файлы, у которых не изменились время и размер; у изменившихся сверяет SHA-1 содержимого. Повторный обход 108 неизменных файлов почти мгновенен.

5.2. По ЛР 2**1. Опишите общий принцип распознавания языка.**

Для каждого языка строится ПОЯ по корпусу, для документа — ПОД по тем же правилам, затем по выбранной метрике ищется ближайший ПОЯ.

2. Назовите три шага по методичке.

Предварительная обработка текста; распознавание — сравнительный анализ профилей; выбор языка с наименьшим значением метрики и представление результатов.

3. Что делается на шаге предобработки HTML?

Удаляются теги и содержимое script/style, раскрываются сущности, текст приводится к строчным буквам, небуквы — к пробелу, ä ö ü ß сохраняются, выполняется нормализация NFC.

4. Что такое закон Зипфа и при чём он здесь?

Частота элемента обратно пропорциональна его рангу. Небольшое число самых частых N-грамм определяется языком, а не темой, поэтому около 300 первых N-грамм — «подпись» языка.

5. Как строится профиль в методе N-грамм?

Слова обрамляются «_», извлекаются все подстроки длиной 1–5, сортируются по убыванию частоты, первые 300 — профиль, номер — ранг.

6. Что такое out-of-place?

Сумма модулей разностей рангов N-грамм документа в профиле документа и профиле языка; отсутствующей в языке N-грамме — максимальный штраф, равный размеру профиля.

7. Зачем обрамлять слово заполнителем?

Чтобы различать начало, середину и конец слова: «_de» и «de_» — разные признаки, а окончания и начала слов очень характерны для языка.

8. Как работает алфавитный метод и где он бессилён?

Сравнивает распределения частот букв манхэттенским расстоянием, делённым на 2. Бессилён для языков с общим алфавитом и для транслитерации: «chisla» он отнесёт к немецкому.

9. Опишите архитектуру нейросети.

400 входов (частоты символьных 1–3-грамм) → скрытый слой 32 нейрона с tanh → 2 выхода с softmax. Обучение — обратное распространение ошибки, Adam, перекрёстная энтропия с L2.

10. Что такое softmax и зачем он на выходе?

Превращает логиты в вероятности: $P_i = \exp(z_i) / \sum \exp(z_i)$, все положительные и в сумме 1. Из вероятности получается расстояние $1 - P$.

11. Что делает оптимизатор Adam?

Для каждого веса хранит скользящие средние градиента и его квадрата с поправкой смещения и делает адаптивный шаг — быстрее и устойчивее обычного градиентного спуска.

12. Почему нейросеть обучали на фрагментах разной длины?

Короткий текст даёт разреженный вектор частот; обученная на длинных фрагментах сеть проседала до 90 % на коротком входе. Обучение на 150, 300 и 600 символах дало 100 % на всех длинах.

13. Что такое утечка данных и как её избежали?

Попадание одних и тех же данных и в обучение, и в контроль. Корпус делится на части по тексту до нарезки на перекрывающиеся фрагменты.

14. Почему все методы дали 100 %? Это хорошо?

Это свойство пары языков: кириллица и латиница не пересекаются. Различия методов видны на смешанном тексте и проявились бы на паре с общим алфавитом.

15. Как устроено голосование?

Большинство голосов трёх методов; при равенстве побеждает метод с наибольшей уверенностью $(d_2 - d_1) / d_2$. Время голосования — сумма времён всех методов.

16. Почему темы тестовых документов попарно совпадают?

Чтобы методы не могли различать документы по теме вместо языка — иначе оценка была бы завышенной.

17. Почему объём корпусов сравнивают по буквам?

В UTF-8 кириллица занимает 2 байта на букву, латиница — 1. По байтам корпуса различаются вдвое, по буквам — всего в 1,06 раза.

18. Где распознавание языка применяется в жизни?

Браузеры (предложение перевода, CLD3 в Chrome), переводчики, поисковые системы (выбор морфологии), фильтрация корпусов для языковых моделей (fastText, 176 языков), маршрутизация обращений.

5.3. Сквозные вопросы**1. Что общего у ЛР 1 и ЛР 2?**

Обе строят поисковые образы и сравнивают их: ПОД с ПОЗ для ранжирования, ПОД с ПОЯ для классификации. Обе проходят один конвейер от извлечения текста до оценки на эталоне.

2. Мера близости или расстояния — в чём разница?

Близость (косинус) тем больше, чем сильнее похожи объекты; расстояние (out-of-place, манхэттенское, $1 - P$) — тем меньше. Любую ограниченную близость можно превратить в расстояние: $d = 1 - \text{sim}$.

3. Зачем в обеих работах отдельная эталонная разметка?

Качество нельзя оценить без правильных ответов, сделанных человеком независимо от системы: релевантность в ЛР 1, язык в ЛР 2.

4. Как устроена архитектура обеих систем?

Данные — ядро алгоритмов — веб-интерфейс на FastAPI + Jinja2, плюс сценарии отчёта в tools/, тесты и отключаемая игровая надстройка, не влияющая на измерения.

Часть 6. Глоссарий

Русский термин — английский эквивалент (для поиска литературы) — кратко.

Термин	English	Кратко
Информационный поиск	information retrieval, IR	поиск неструктурированных документов под информационную потребность
Информационная потребность	information need	что человек на самом деле хочет узнать
Релевантность	relevance	соответствие документа потребности
Поисковый образ документа (ПОД)	document representation	документ как набор ключевых слов с весами
Поисковый образ запроса (ПОЗ)	query representation	запрос в той же форме, что ПОД
Поисковый образ языка (ПОЯ)	language profile	профиль языка по обучающему корпусу
Паук, кроулер	crawler, spider	агент, собирающий документы
Инвертированный индекс	inverted index	термин → список документов с весами
Термин, словарь	term, vocabulary	единица индекса и их множество
Стоп-слова	stop words	служебные слова, исключаемые из индекса
Лемма, лемматизация	lemma, lemmatization	словарная форма слова и приведение к ней
Стемминг	stemming	отсечение окончаний по правилам
Частота термина	term frequency, TF	Q_i
Документная частота	document frequency, DF	P_i
Инверсная частота	inverse document frequency, IDF	$B_i = \log(N / P_i)$
Векторная модель	vector space model	документы и запросы — векторы весов
Косинусная мера	cosine similarity	косинус угла между векторами
Булев поиск	Boolean retrieval	логическая стратегия: И, ИЛИ, НЕ
BM25	Okapi BM25	вероятностная функция ранжирования
Обратная связь по релевантности	relevance feedback	уточнение запроса по отметкам пользователя
Метод Рокчио	Rocchio algorithm	сдвиг вектора запроса к удачным документам
Расширение запроса	query expansion	добавление синонимов и связанных терминов
Тезаурус	thesaurus	словарь связей между терминами
Расстояние Дамерау — Левенштейна	Damerau–Levenshtein distance	число правок, включая перестановку соседних символов
Сниппет	snippet	фрагмент документа в выдаче
Эталонная разметка	relevance judgments, qrels; ground truth	правильные ответы, сделанные человеком
Пул	pool	документы, просмотренные экспертом
Точность / полнота	precision / recall	доля релевантных среди найденных / найденных среди релевантных
F-мера	F-measure, F1	гармоническое среднее P и R
Средняя точность	average precision, AP; MAP	точность в точках попаданий; среднее по запросам

Термин	English	Кратко
nDCG	normalized discounted cumulative gain	выигрыш с градациями и дисконтом по позиции
bpref	binary preference	метрика, устойчивая к неполной разметке
Распознавание языка	language identification, LID	определение языка текста
N-грамма	character n-gram	подстрока из N символов
Закон Зипфа	Zipf's law	частота обратно пропорциональна рангу
Мера несовпадения позиций	out-of-place measure	сумма разностей рангов N-грамм
Манхэттенское расстояние	Manhattan distance, L1	сумма модулей разностей координат
Персептрон	multilayer perceptron, MLP	полносвязная нейросеть
Функция активации	activation function (tanh, ReLU, softmax)	нелинейность нейрона
Перекры́стная энтропия	cross-entropy loss	функция потерь классификации
Обратное распространение ошибки	backpropagation	вычисление градиентов по цепному правилу
Оптимизатор Adam	Adam optimizer	градиентный спуск с адаптивным шагом
Регуляризация L2	L2 regularization, weight decay	штраф за большие веса
Эпоха, мини-батч	epoch, mini-batch	проход по данным; порция примеров на один шаг
Утечка данных	data leakage	пересечение обучающих и контрольных данных
Матрица ошибок	confusion matrix	таблица «эталон × ответ»
Доля верных ответов	accuracy	верных / всего
Нормализация Unicode (NFC)	Unicode normalization form C	единая составная запись символов

Источники

- Методические указания к лабораторным работам «Разработка информационно-поисковой системы. Реализация методов оценки качества её работы» и «Автоматическое распознавание языка текста».
- Отчёт по ЛР 1 (ИПС «Арахна», вариант 3), README и материалы для отчёта; README и протокол результатов ЛР 2 («Толмач», вариант 4); исходный код обеих систем.
- Savnar W. B., Trenkle J. M. N-Gram-Based Text Categorization, 1994.
- Российский семинар по оценке методов информационного поиска: romip.narod.ru.
- Google CLD3: github.com/google/cld3.
- fastText, language identification: fasttext.cc/docs/en/language-identification.html.
- Apache Lucene, BM25Similarity: lucene.apache.org.